HU-01 · Cierre del cuestionario adaptativo

> *Como Oficial de Cumplimiento o Evaluador de IA quiero responder un cuestionario adaptativo con saltos lógicos condicionales organizados por dimensiones, para contestar únicamente las preguntas relevantes según el perfil de la organización.*

Estado previo (revisión del 15-oct): C1, C2 y C3 cumplidos; **C4 no** (`GET controles-aplicables` p95 542 ms) y un defecto de backend (respuestas duplicadas, sin validar aplicabilidad). Este cambio cubre las 4 tareas pendientes.

Criterio	Estado	Evidencia
C1 · 4 dimensiones	☐ (ya cumplía)	`GET /progreso` además desglosa el avance por dimensión
C2 · Saltos condicionales	☐ reforzado	Ahora la API **rechaza** responder controles que el filtro ocultó (422)
C3 · Progreso en PostgreSQL	☐ reforzado	Upsert por (evaluación, control) + `GET /progreso` para reanudar
C4 · Latencia < 500 ms	☐	`controles-aplicables` p95 542 → **24,1 ms** (reporte oficial `reporte_latencia_live_hu01.md`, 12/12 endpoints)

1. Una respuesta por control (upsert + restricción única)

- **Migración** `9a4e2c7b1f05` (después de `f7b2d1a8c3e9`): fusiona los duplicados que ya existan —conserva la respuesta más reciente de cada (evaluación, control) y le traspasa evidencias, eventos de seguridad, llamadas al LLM y sanitizaciones de las demás— y crea `UNIQUE ("Evaluacion_id", "Matriz_controles_id_control")` (`uq_respuesta_evaluacion_control`). Probada en PostgreSQL 16 con 80 filas de 40 controles (quedaron 40, la evidencia se traspasó) y con `downgrade -1` / `upgrade head`.
- **`POST /api/respuestas/` es un upsert**: 201 la primera vez, **200** si el control ya estaba respondido (mismo `id_respuesta`). Cuerpo: `{id_respuesta, creada, actualizada}`.
  - Reenvío idéntico (guardado automático) → no toca la fila, así no se pierde la validación del LLM.
  - Declaración distinta → pasa a ser el estado vigente (`fuente = usuario`); se descarta la validación anterior (confianza, revisión humana) porque era de otra declaración; la brecha se asigna o se quita según sea `no_cumplido`. Las evidencias subidas se conservan.
  - Carrera: si dos peticiones insertan el mismo control a la vez, la BD rechaza la segunda y el caso de uso actualiza la fila ganadora (probado con dos sesiones reales en PostgreSQL y con 10 POST simultáneos contra la API: 1 × 201, 9 × 200, 1 fila).
- **`PUT /api/respuestas/{id}`** `{estado_clasificacion, justificacion}`: modifica por id, con las mismas validaciones (incluida la capa anti prompt-injection).
- El resumen (`/cerrar`) cuenta **una respuesta por control** también por defensa en el código.

- `test_riesgo_respuestas_duplicadas_de_un_control_cuentan_doble` pasó a `test_respuestas_duplicadas_de_un_control_cuentan_una_sola_vez` (esperado: 1 control y 100 %, antes 2 controles y 50 %) + `test_la_tabla_respuestas_tiene_unique_evaluacion_control`.

2. Rechazo de respuestas no válidas

Validado en `CreateRespuestaUseCase` (POST y PUT), en este orden. Errores con cuerpo `{"detail": {"codigo": ..., "mensaje": ...}}`:

Caso	HTTP	`codigo`
Clasificación fuera de las 4 declarables	400	(texto, como antes)
Evaluación inexistente	404	`EVALUACION_NO_ENCONTRADA`
Evaluación cerrada (`estado_progreso` = `completada` o `cerrada`)	409	`EVALUACION_CERRADA`
Filtro F1–F7 sin completar	409	`FILTRO_PENDIENTE`
Control inexistente	404	`CONTROL_NO_ENCONTRADO`
Control que el filtro no deja aplicable	422	`CONTROL_NO_APLICABLE`
Justificación sospechosa	422	`PROMPT_INJECTION` (como antes)
PUT a una respuesta inexistente	404	`RESPUESTA_NO_ENCONTRADA`

Además, una evaluación cerrada **no permite rehacer el filtro** (`POST /filtro` → 400).

> *La evaluación se cierra con `POST /api/evaluaciones/{id}/completar` (sección 5).*

3. Optimización de `GET /evaluaciones/{id}/controles-aplicables` (C4)

Causa: por petición hacía ~6 consultas (evaluación; filtro + preguntas; controles + 3 por las relaciones `lazy="selectin"` dominio/brecha/ruta) y validaba 22–40 objetos ORM con Pydantic.

- `EvaluacionRepository.get_contexto()`: evaluación + respuestas del filtro en **una** consulta (LEFT JOIN, solo columnas). La usan también el POST de respuestas y el progreso.
- `app/infrastructure/cache/catalogo_controles.py`: el catálogo de controles (dato semilla) se carga

  una vez, se serializa con el **mismo** `ControlResponse` y se guarda el JSON ya codificado de cada combinación de controles (máx. 64). TTL 5 min (`CATALOGO_TTL_SEGUNDOS`) e `invalidar()` para el futuro mantenedor de la matriz (HU-24). Una prueba compara la salida con la de la versión anterior.
- `RespuestaRepository.get_control()` ya no carga dominio/brecha/ruta (no se usaban): el POST de

respuestas hace más validaciones y aun así queda en la misma latencia que antes.

Medición de referencia (`tests/test_latencia_api.py` live, concurrencia 10, 50 muestras; uvicorn + PostgreSQL 16 + Redis reales en Linux de 2 núcleos, driver psycopg). Misma máquina antes y después:

Endpoint	p95 antes	p95 después (3 corridas)
GET controles aplicables	152 ms	**24 / 29 / 35 ms**
POST respuesta (ahora upsert + validaciones)	86 ms	82 / 101 / 84 ms
GET progreso (nuevo)	—	51 / 42 / 46 ms

Con concurrencia 25 (100 muestras) todos los endpoints siguen bajo 250 ms (controles-aplicables p95 92 ms). **Medición oficial** (16-oct-2026, Docker sobre WSL2, concurrencia 10, 50 muestras; `reporte_latencia_live_hu01.md`): **12/12 endpoints cumplen**. `GET controles-aplicables` p95 **24,1 ms** (antes 542,7 ms), `POST respuesta` 67,2 ms, `GET progreso` 43,7 ms; el más lento fue `GET filtro` con 52,1 ms.

4. Progreso para reanudar: `GET /api/evaluaciones/{id}/progreso`


```
{
    "evaluacion_id": 9001, "estado_progreso": "en_progreso", "cerrada": false,
    "filtro_completado": true, "etapa": "cuestionario",
    "controles_aplicables": 40, "controles_respondidos": 4, "controles_pendientes": 36,
    "porcentaje": 10.0, "respuestas_fuera_de_alcance": 0,
    "por_dimension": [{"dominio_id": 1, "dominio": "Protección de Datos", "aplicables":
  10, "respondidos": 1, "porcentaje": 10.0}, "..."],
    "siguiente_control": {"id_control": 2, "codigo_control": "DAT-002", "dominio_id":
  1, "dominio": "Protección de Datos"},
    "pendientes": ["... controles en el mismo orden que controles-aplicables ..."],
    "respondidos": [{"id_control": 1, "codigo_control": "DAT-001", "id_respuesta": 15,
  "estado_clasificacion": "cumplido", "estado_clasificacion_final": "cumplido", "...":
  "..."}]
  }
```


- `etapa`: `filtro` (falta F1–F7) · `cuestionario` · `completo` · `cerrada`.
- Solo cuentan respuestas de controles **aplicables hoy**; si se rehízo el filtro, las demás van a `respuestas_fuera_de_alcance` (tampoco puntúan en `/cerrar`, como ya ocurría).
- No expone justificaciones (texto libre con posible PII): para el detalle está `GET /api/respuestas/{id}`.
- 2 consultas por petición (contexto + estados de las respuestas) más el catálogo cacheado.

5. Ampliación: completar la evaluación y bloquear evidencias

`POST /api/evaluaciones/{id}/completar`: cierre definitivo. `POST /cerrar` sigue igual (recalcula el resumen como vista previa, no cambia el estado y se puede repetir).

Condición	Si no se cumple
La evaluación existe	404 `EVALUACION_NO_ENCONTRADA`
No está ya cerrada	409 `EVALUACION_CERRADA`
Filtro F1–F7 completo	409 `FILTRO_PENDIENTE`
Todos los controles aplicables respondidos	409 `CUESTIONARIO_INCOMPLETO` ("Faltan N de M...: códigos")
Ninguna evidencia `pending`/`processing`	409 `EVIDENCIAS_EN_PROCESO` (su resultado llegaría después del cierre)

Si todo se cumple, genera el resumen y fija `estado_progreso = 'completada'` en la **misma transacción**, y responde el resumen más `estado_progreso` y `controles_aplicables`. La fila de la evaluación se bloquea (`SELECT … FOR UPDATE`): con 5 llamadas simultáneas contra PostgreSQL, una obtiene 200 y las otras 409. Con F1 = No se puede completar (0 controles, sin puntaje).
Desde ahí la evaluación entra al benchmark (el cron solo cuenta las `completada`; hallazgo #4 resuelto) y **no admite**: respuestas nuevas ni cambios (409), rehacer el filtro (400) ni **evidencias**: `POST /api/respuestas/{id}/evidencias` → 409 `EVALUACION_CERRADA`, sin guardar el archivo. El bloqueo de evidencias vale para `completada` y `cerrada`.

Archivos: `app/application/use_cases/completar_evaluacion.py` (nuevo), `app/api/routes/reportes.py`, `app/application/use_cases/upload_evidence.py`, `app/api/routes/evidencias.py`, `tests/test_completar_evaluacion.py` (nuevo, 9 pruebas), `tests/test_bordes_na_y_evidencia.py`.

No hay forma de **reabrir** una evaluación completada (decisión pendiente del PO).

Archivos

Nuevos

- `migrations/versions/9a4e2c7b1f05_hu01_respuesta_unica_por_control.py`
- `app/infrastructure/cache/catalogo_controles.py`
- `app/application/use_cases/progreso_cuestionario.py`
- `tests/test_hu01_cuestionario.py` (22 pruebas: 21 en proceso + 1 de carrera solo live)
- `CAMBIOS_HU01.md`

Modificados

- `app/domain/models/respuesta.py` (UniqueConstraint), `app/domain/models/evaluacion.py` (`ESTADOS_CERRADOS`, `esta_cerrada`)
- `app/application/use_cases/create_respuesta.py` (upsert, PUT, validaciones, errores tipados)

- `app/infrastructure/repositories/evaluacion_repository.py` (`ContextoEvaluacion`, `get_contexto`)
- `app/infrastructure/repositories/respuesta_repository.py` (`get_by_evaluacion_control`, `list_estado_por_evaluacion`, `RespuestaDuplicadaError`, `get_control` sin relaciones)
- `app/api/routes/respuestas.py` (POST upsert, PUT), `app/api/routes/filtro.py` (controles-aplicables optimizado, `/progreso`)
- `app/application/schemas/api_schemas.py` (esquemas de progreso)
- `app/application/use_cases/generate_resumen.py` (una respuesta por control), `app/application/use_cases/responder_filtro.py` (no rehacer filtro si está cerrada)
- Pruebas: `conftest.py` (invalida la caché por test), `test_bordes_na_y_evidencia.py`, `test_seguridad_integracion.py` (fakes con contexto de evaluación), `test_e2e_capa_ia.py`, `test_solicitudes_arcop.py`, `test_live_capa_ia.py` (completan el filtro antes de responder), `test_latencia_api.py` y `test_defectos_api.py` (aceptan 200 del upsert; la de latencia mide también `/progreso`)
- `../README.md` (reglas del cuestionario, endpoints, hallazgo #7 resuelto)

Cómo aplicarlo y verificar (Docker)

```
# desde Proyecto/
  docker compose up -d --build                        # 'migrate' aplica 9a4e2c7b1f05
  docker compose exec api alembic current             # debe mostrar 9a4e2c7b1f05 (head)
  docker compose exec worker pytest tests -v -rxX     # suite completa

  # Latencia live (evidencia oficial de C4). Crea/limpia las evaluaciones 9001 y 9002.
  docker compose exec api sh -c 'LATENCIA_BASE_URL=http://localhost:8000 LATENCIA_DATAB
  ASE_URL=$DATABASE_URL LATENCIA_CONCURRENCIA=10 LATENCIA_N=50 LATENCIA_REPORTE=reporte
  _latencia_live_hu01.md pytest tests/test_latencia_api.py -v -s'

  # Carrera real de 10 POST simultáneos al mismo control (PostgreSQL)
  docker compose exec api sh -c 'LATENCIA_BASE_URL=http://localhost:8000 LATENCIA_DATAB
  ASE_URL=$DATABASE_URL pytest tests/test_hu01_cuestionario.py -v -k live'
```

Resultado en el entorno de desarrollo de esta entrega (sin spaCy instalado, por eso las pruebas de NER quedan omitidas): **245 aprobadas, 8 omitidas**; live contra PostgreSQL: latencia 12/12 endpoints y la prueba de carrera aprobada 6 de 6 veces.

Impacto para el frontend (Sprint 4)

- `POST /api/respuestas/` puede devolver **200** además de 201: tratar ambos como éxito.
- Antes de responder controles hay que completar el filtro; usar `GET /progreso` al abrir una

evaluación para saber la `etapa` y el `siguiente_control`.

Fuera de alcance / pendiente

- Reabrir una evaluación completada (decisión del PO).
- Autorización por sesión del encuestado (HU-20): hoy cualquiera puede responder cualquier evaluación.
- Crear evaluaciones por API (HU-19).